

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	KOTHAKOTA RAKESH
Reg. No	192525075

COURSE NAME: Deep Learning
FACULTY: Dr. Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

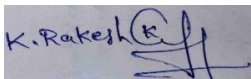
Duration: 30 minutes

Marks: 25

Code of Conduct:

I **Kothakota Rakesh /192525075** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: K.Rakesh

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer: In gradient descent, the learning rate controls the size of the parameter update:

$$w(t+1) = w(t) - \alpha * \text{gradient}$$

where α is the learning rate. If the loss function is convex, there is only one global minimum, so the main problem is not getting trapped in local minima, but reaching the minimum efficiently and stably.

Using a **high initial learning rate followed by gradual decay** can be useful because, in the beginning, the model parameters are usually far from the optimum. A large learning rate allows the algorithm to take bigger steps and move quickly toward the region of minimum loss. This can reduce training time compared with using a small constant learning rate from the start.

As training progresses, the parameters come closer to the optimum. At this stage, a high learning rate may cause the updates to overshoot the minimum or oscillate around it. Therefore, decreasing the learning rate later helps the algorithm make finer adjustments. This improves stability and allows the model to converge more accurately.

For example, with a constant high learning rate, the loss may decrease quickly at first but may not settle near the minimum. With a constant low learning rate, convergence may be stable but very slow. A decaying learning rate combines both advantages: fast progress initially and stable convergence later.

This strategy can outperform a constant learning rate under certain conditions:

1. The initial learning rate is not so high that it causes divergence.
2. The loss function is smooth, meaning gradients do not change too abruptly.
3. The decay schedule is chosen properly, such as step decay, exponential decay, or inverse time decay.
4. The problem has noisy gradients, as in stochastic gradient descent, where reducing the learning rate helps reduce fluctuations near the optimum.
5. The optimization landscape has flat regions where a small constant learning rate would move too slowly.

In convex optimization theory, convergence is often guaranteed when the learning rate decreases in a controlled way. For stochastic gradient descent, a common condition is:

$$\sum(\alpha_t) = \text{infinity}$$

$$\sum(\alpha_t^2) < \text{infinity}$$

This means the learning rate should not decrease too quickly, otherwise learning may stop before reaching the optimum. At the same time, it should decrease enough to reduce noise and oscillations.

However, this strategy also has risks. If the initial learning rate is too large, the algorithm may overshoot the minimum repeatedly, causing the loss to oscillate or even increase. In extreme cases, the training may diverge completely. Another risk is decaying the learning rate too early or too fast. Then the model may make very small updates before reaching the optimum, causing slow convergence or premature stopping. Also, if the decay schedule is not suitable for the dataset or model, performance may be worse than a carefully chosen constant learning rate.

For a convex loss function with Lipschitz-continuous gradients, the learning rate usually needs to stay below a stability limit related to the curvature of the loss surface. If the learning rate is larger than this limit, gradient descent becomes unstable. Therefore, a high initial learning rate is beneficial only when it is high enough to speed up learning but not so high that it breaks convergence.

In conclusion, a high initial learning rate followed by decay is an effective optimization strategy because it balances speed and accuracy. It helps the model move rapidly toward the optimum in the early stage and then converge smoothly in the later stage. It can outperform a constant learning rate when the decay schedule is well designed and the initial learning rate remains within a stable range. But if chosen poorly, it may cause divergence, oscillations, or slow final convergence.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

Answer:

- The dataset contains outliers
- The sample size increases
What does this imply about the robustness of MLE?
- For a dataset $(x_1, x_2, \dots, x_n)$ generated independently from a Gaussian distribution $(N(\mu, \sigma^2))$, where (μ) is unknown and (σ^2) is known, the likelihood function is formed by multiplying the probability densities of all observations. The Gaussian density is based on the term $((x_i - \mu)^2)$, so maximizing the likelihood is equivalent to minimizing the total squared distance between the observations and (μ) . Taking the log-likelihood and differentiating with respect to (μ) , we get:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

Thus, the Maximum Likelihood Estimator for the unknown mean (μ) is the sample mean. When the dataset contains outliers, the MLE is strongly affected because the sample mean uses every data value equally. An extremely large or small value pulls the mean toward itself. This effect is even stronger because the Gaussian likelihood contains squared error terms, so large deviations receive high importance. Therefore, if the data has outliers, the estimated mean may shift away from the true central value of most data points.

As the sample size increases, the estimator becomes more accurate if the data truly follows a Gaussian distribution. By the Law of Large Numbers, the sample mean approaches the true mean as (n) becomes large. Also, the variance of the estimator decreases as (σ^2/n) , meaning the estimate

becomes more stable. However, if outliers continue to appear regularly in the data, increasing sample size alone may not completely remove their effect.

This shows that MLE is efficient and consistent when the assumed Gaussian model is correct, but it is not robust to outliers. Since the MLE for μ is the sample mean, extreme observations can significantly change the estimate. In such cases, robust methods like median, trimmed mean, or heavy-tailed distributions may give better results.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer: When a dataset has 10,000 features but only 500 training samples, the main challenge is the **high-dimensional, low-sample-size problem**. Since the number of features is much larger than the number of samples, the model can easily memorize the training data instead of learning useful patterns. This causes overfitting, where training accuracy is high but test accuracy is poor. Another issue is that many features may be irrelevant, redundant, or noisy. Distance-based methods may also perform badly because of the curse of dimensionality, where data points become sparse in high-dimensional space.

A suitable pipeline begins with **data preprocessing**. Missing values should be handled, categorical values should be encoded if present, and all numerical features should be standardized. Standardization is important because algorithms such as logistic regression, SVM, PCA, and regularized models are sensitive to feature scale. After preprocessing, the dataset should be split using cross-validation because only 500 samples are available, and a single train-test split may give an unreliable estimate of performance.

Next, **feature selection** should be applied to remove irrelevant and noisy features. Simple filter methods such as correlation, chi-square test, mutual information, or ANOVA can be used to select the most informative features. This reduces the input size and helps the model focus on useful variables. After that, dimensionality reduction methods such as PCA may be used to transform the selected features into a smaller number of components while preserving most of the variance. This helps reduce noise and computational cost, but PCA should be fitted only on the training data inside cross-validation to avoid data leakage.

For the classifier, a regularized model such as **L1/L2 logistic regression, linear SVM, or elastic net** is suitable. L1 regularization helps by forcing many feature weights to zero, performing automatic feature selection. L2 regularization penalizes large weights and makes the model smoother. Elastic net combines both benefits. Hyperparameters such as regularization strength should be selected using cross-validation. It is better to avoid highly complex models such as deep neural networks unless more data is available, because they can overfit severely.

This approach mitigates overfitting by reducing the number of effective features, removing noise, and restricting model complexity. Feature selection removes irrelevant inputs, PCA compresses the data into fewer dimensions, and regularization prevents the model from assigning excessively large weights to features. Cross-validation gives a more reliable estimate of generalization performance. Thus, the pipeline converts a high-risk learning problem into a more stable classification task.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer: A Multilayer Perceptron with one hidden layer computes the output by first applying weighted sums and sigmoid activations in the hidden layer. Each hidden neuron produces a nonlinear transformation of the input. When more hidden neurons are added, the network gets more adjustable weights and biases. This increases its capacity to represent more complex functions. Analytically, a one-hidden-layer MLP can be written as a weighted sum of sigmoid functions, so increasing the number of hidden neurons increases the number of nonlinear basis functions available to approximate the target function.

Because sigmoid functions are nonlinear, each hidden neuron can learn a different boundary or feature pattern in the input space. With only a few hidden neurons, the network may represent only simple decision boundaries and may underfit the data. By adding more neurons, the MLP can combine many nonlinear transformations and form more flexible decision regions. In theory, according to the universal approximation idea, a sufficiently large hidden layer can approximate many continuous functions to any desired accuracy.

However, adding more hidden neurons does not always improve performance. More neurons also mean more parameters. If the training dataset is small or noisy, the network may start memorizing the training examples instead of learning general patterns. This leads to overfitting, where training error becomes very low but test error remains high. In such a case, the model has high representational power but poor generalization.

Better performance depends not only on model capacity but also on the amount of data, noise level, optimization, and regularization. A very large hidden layer may also make training slower and more sensitive to initialization. Sigmoid activations can suffer from vanishing gradients when neurons saturate near 0 or 1, which makes learning difficult. Therefore, more hidden neurons are useful only when the extra capacity matches the complexity of the problem.

In conclusion, increasing hidden neurons improves the representational power of an MLP because the network can combine more nonlinear sigmoid functions and learn more complex mappings. But this does not guarantee better performance because excessive capacity can cause overfitting and weak generalization. Techniques like validation, early stopping, weight decay, dropout, and choosing an appropriate network size help balance representation and generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

In a very high-dimensional dataset, the **curse of dimensionality** means that data becomes sparse and the number of possible feature combinations becomes very large. During backpropagation, gradient updates are computed with respect to many weights and features. If many features are irrelevant or noisy, their gradient components may also be noisy. Therefore, the update direction in SGD may not point clearly toward the minimum. In high dimensions, gradients can also become poorly scaled across different directions, so some parameters change too much while others change very slowly.

The convergence speed of SGD is also affected. Since SGD uses a small batch or single sample to estimate the gradient, the gradient estimate already contains noise. In high-dimensional spaces, this noise can increase because many unnecessary dimensions contribute to the update. As a result, SGD may take more iterations to converge, oscillate around minima, or move slowly in flat directions. Poor conditioning of the loss surface can make this worse, because the algorithm may progress fast in steep directions but very slowly in shallow directions.

High dimensionality also harms **generalization**. If the number of features or model parameters is very large compared to the number of training samples, the network can memorize training data instead of learning meaningful patterns. This causes overfitting: training loss becomes low, but test performance is poor. Analytically, model complexity increases with the number of parameters, and generalization error bounds usually become worse when complexity is high and sample size is limited. Thus, more dimensions require more data to learn reliably.

To overcome these issues, one technique is **dimensionality reduction**, such as PCA or an autoencoder. PCA reduces the original feature dimension from d to a smaller value k , while preserving important variance. This removes noise, reduces computation, and makes gradient updates more stable. Another technique is **regularization**, such as L1, L2, dropout, or early stopping. L2 regularization adds a penalty $(\lambda \|w\|^2)$, which prevents weights from becoming too large and improves generalization. L1 regularization encourages sparsity by pushing irrelevant feature weights toward zero. Dropout prevents the network from depending too much on specific neurons. Together, dimensionality reduction and regularization reduce effective model complexity, improve convergence, and help the model generalize better.